Stacks in Java — All Java Codes

Original Java source-code presentation: monospace code, syntax highlighting, and separate file names. All 15 examples are included.

1. Push and print a stack

Main.java

```java
import java.util.Stack;

public class Main {
    public static void main(String[] args) {
        Stack<Integer> s1 = new Stack<Integer>();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        System.out.println(s1.toString());
    }
}
```

2. peek() looks, pop() takes

Main.java

```java
import java.util.Stack;

public class Main {
    public static void main(String[] args) {
        Stack<Integer> s1 = new Stack<Integer>();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        System.out.println("Original Stack: " + s1.toString());

          int top = s1.peek();
          System.out.println("top element: " + top);
          System.out.println("Stack after accessing top element: " + s1);

          int out = s1.pop();
          System.out.println("Removed element: " + out);
          System.out.println("Stack after Pop Operation: " + s1);
      }
}
```

3. Generics: one class, many types

Main.java

```java
class Box<T> {
    private T item;
    public void set(T item) { this.item = item; }
    public T get()          { return item; }
}

public class Main {
    public static void main(String[] args) {
        Box<Integer> box1 = new Box<Integer>();             // only Integers
        box1.set(25);
        int age = box1.get();                               // no cast needed

          Box<String> box2 = new Box<String>();             // only Strings
          box2.set("Ali");
          String name = box2.get();

          // box1.set("Ali");   // COMPILE ERROR: String is not an Integer

          System.out.println("box1 holds " + age);
          System.out.println("box2 holds " + name);
      }
}
```

4. Bank: undo the last deposit

UndoDepositDemo.java

```
import java.util.Stack;

 class BankAccount {
     private double balance;
     private Stack<Double> history = new Stack<>();

       public BankAccount(double balance) { this.balance = balance; }

       public void deposit(double amount) {
           balance = balance + amount;
           history.push(amount);                // remember it
           System.out.println("Deposit " + amount + " -> balance " + balance);
       }

       public void undoLastDeposit() {
           if (history.isEmpty()) {
               System.out.println("Nothing to undo");
               return;
           }
           double last = history.pop();          // most recent deposit
           balance = balance - last;
           System.out.println("Undo " + last + " -> balance " + balance);
       }
 }

 public class UndoDepositDemo {
     public static void main(String[] args) {
         BankAccount acc = new BankAccount(100);
         acc.deposit(50);
         acc.deposit(20);
         acc.deposit(30);
         acc.undoLastDeposit();
         acc.undoLastDeposit();
     }
 }
```

5. Student: undo the last course

UndoCourseDemo.java

```
import java.util.Stack;

 class Registration {


       private String studentName;
       private Stack<String> courses = new Stack<>();

       public Registration(String studentName) { this.studentName = studentName; }

       public void addCourse(String code) {
           courses.push(code);
           System.out.println("Added " + code + " -> " + courses);
       }

       public void undoLastCourse() {
           if (courses.isEmpty()) {
               System.out.println("Nothing to undo");
               return;
           }
           String removed = courses.pop();
           System.out.println("Removed " + removed + " -> " + courses);
       }
 }

 public class UndoCourseDemo {
     public static void main(String[] args) {
         Registration reg = new Registration("Hodan");
         reg.addCourse("CA314");
         reg.addCourse("CA301");
         reg.addCourse("MA210");
         reg.undoLastCourse();
     }
 }
```

6. Reversing a string

Reverser.java

```
import java.util.Stack;

 public class Reverser {
     public String reverse(String input) {
         Stack<Character> stack = new Stack<>();
         for (int i = 0; i < input.length(); i++)
             stack.push(input.charAt(i));       // J, A, V, A

           String reversed = "";
           while (!stack.isEmpty())
               reversed += stack.pop();                  // A, V, A, J
           return reversed;
       }
 }
```

Main.java

```
import java.util.Scanner;

 public class Main {
     public static void main(String[] args) {
         Scanner inputObject = new Scanner(System.in);
         System.out.println("Enter String to Reverse: ");

           String input = inputObject.nextLine();
           Reverser test = new Reverser();
           String result = test.reverse(input);
           System.out.println("Reversed String: " + result);
       }
}
```

7. ArrayStack: full implementation

ArrayStack.java

```
import java.util.Arrays;
 import java.util.EmptyStackException;

 public class ArrayStack<T> {
     private final static int DEFAULT_CAPACITY = 2;   // constant
     private int top;          // next free slot = number of elements
     private T[] stack;

       public ArrayStack() {                        // empty stack, default capacity
           this(DEFAULT_CAPACITY);
       }

       public ArrayStack(int initialCapacity) { // empty stack, given capacity
           top = 0;
           stack = (T[]) (new Object[initialCapacity]); // generic array trick
       }

       public void push(T element) {
           if (size() == stack.length)
               expandCapacity();                    // full? double the array
           stack[top] = element;
           top++;
       }

       private void expandCapacity() {
           stack = Arrays.copyOf(stack, stack.length * 2);
       }

       public T pop() throws EmptyStackException {
           if (isEmpty())
               throw new EmptyStackException(); // underflow
           top--;
           T result = stack[top];
           stack[top] = null;
           return result;
       }

       public T peek() throws EmptyStackException {
           if (isEmpty())
               throw new EmptyStackException();
           return stack[top - 1];               // look only, do not remove
       }

       public int size()          { return top; }
       public boolean isEmpty()   { return top == 0; }
       public int capacity()      { return stack.length; }              // added for the trace
 }
```

8. ArrayStack trace

ArrayStackTraceDemo.java

```
public class ArrayStackTraceDemo {
     public static void main(String[] args) {
         ArrayStack<String> s = new ArrayStack<>();
         s.push("A");
         s.push("B");
         System.out.println("size " + s.size() + ", capacity " + s.capacity());
         s.push("C");                        // array full -> doubles to 4
         System.out.println("size " + s.size() + ", capacity " + s.capacity());

           System.out.println("peek: " + s.peek());
           System.out.println("pop: " + s.pop());
           System.out.println("pop: " + s.pop());
           System.out.println("pop: " + s.pop());
           System.out.println("pop: " + s.pop());             // stack empty -> underflow
       }
 }
```

9. Defining our own exception

EmptyCollectionException.java

```
public class EmptyCollectionException extends RuntimeException {
     public EmptyCollectionException(String collection) {
         super("The " + collection + " is empty.");
     }
 }
```

Main.java

```
import java.util.Stack;

 public class Main {
     public static void main(String[] args) {
         Stack<Integer> s1 = new Stack<Integer>();
         s1.push(10);
         s1.push(20);
         s1.push(30);
         System.out.println(s1.toString());
     }
 }

 Output

 [10, 20, 30]

     toString() prints from bottom to top. The top is the right-most value (30).
     Common mistake: expecting [30, 20, 10] . Printing does not remove anything.

 Example 2      peek() looks, pop() takes
Show the difference between peek() and pop() .

 Main.java

 import java.util.Stack;

 public class Main {
     public static void main(String[] args) {
         Stack<Integer> s1 = new Stack<Integer>();
         s1.push(10);
         s1.push(20);
         s1.push(30);
         System.out.println("Original Stack: " + s1.toString());

           int top = s1.peek();
           System.out.println("top element: " + top);
           System.out.println("Stack after accessing top element: " + s1);

           int out = s1.pop();
           System.out.println("Removed element: " + out);
           System.out.println("Stack after Pop Operation: " + s1);
       }
 }

 Output

 Original Stack: [10, 20, 30]
 top element: 30
 Stack after accessing top element: [10, 20, 30]
 Removed element: 30
 Stack after Pop Operation: [10, 20]

     Line 3 still shows 30: peek() only examines the top.


     pop() removes the top and returns it.

 Example 3      Generics: one class, many types
A generic Box<T> that can hold Integers or Strings, with type safety. (The slides show only the
header and the two declarations; this is a minimal complete version.)

 Main.java

 class Box<T> {
     private T item;
     public void set(T item) { this.item = item; }
     public T get()          { return item; }
 }

 public class Main {
     public static void main(String[] args) {
         Box<Integer> box1 = new Box<Integer>();            // only Integers
```

```
        box1.set(25);
        int age = box1.get();                              // no cast needed

          Box<String> box2 = new Box<String>();            // only Strings
          box2.set("Ali");
          String name = box2.get();

          // box1.set("Ali");   // COMPILE ERROR: String is not an Integer

          System.out.println("box1 holds " + age);
          System.out.println("box2 holds " + name);
      }
}
```

Output

```
box1 holds 25
box2 holds Ali
```

T (Type) or E (Element) is the placeholder name by convention.
The compiler rejects the wrong type, so mistakes are caught before the program runs.

Part B — Stack applications

Example 4 Bank: undo the last deposit

Each deposit is pushed onto a history stack; undo pops the most recent one.

UndoDepositDemo.java

```
import java.util.Stack;

class BankAccount {
    private double balance;
    private Stack<Double> history = new Stack<>();

      public BankAccount(double balance) { this.balance = balance; }

      public void deposit(double amount) {
          balance = balance + amount;
          history.push(amount);                // remember it
          System.out.println("Deposit " + amount + " -> balance " + balance);
      }

      public void undoLastDeposit() {
          if (history.isEmpty()) {
              System.out.println("Nothing to undo");
              return;
          }
          double last = history.pop();         // most recent deposit
          balance = balance - last;
          System.out.println("Undo " + last + " -> balance " + balance);
      }
}

public class UndoDepositDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(100);
        acc.deposit(50);
        acc.deposit(20);
        acc.deposit(30);
        acc.undoLastDeposit();
        acc.undoLastDeposit();
    }
}
```

Output

```
Deposit 50.0 -> balance 150.0
Deposit 20.0 -> balance 170.0
Deposit 30.0 -> balance 200.0
Undo 30.0 -> balance 170.0
Undo 20.0 -> balance 150.0
```

LIFO: the last deposit (30) is undone first, then 20.
Always check isEmpty() before pop() to avoid underflow.

Example 5 Student: undo the last course

Course registration on the JUST portal: same pattern, but with Strings.

UndoCourseDemo.java

```
import java.util.Stack;
class Registration {


      private String studentName;
      private Stack<String> courses = new Stack<>();
      public Registration(String studentName) { this.studentName = studentName; }
      public void addCourse(String code) {
          courses.push(code);
          System.out.println("Added " + code + " -> " + courses);
      }
      public void undoLastCourse() {
          if (courses.isEmpty()) {
              System.out.println("Nothing to undo");
              return;
          }
          String removed = courses.pop();
          System.out.println("Removed " + removed + " -> " + courses);
      }
}
public class UndoCourseDemo {
    public static void main(String[] args) {
        Registration reg = new Registration("Hodan");
        reg.addCourse("CA314");
        reg.addCourse("CA301");
        reg.addCourse("MA210");
        reg.undoLastCourse();
    }
}
```

```
Output
Added CA314 -> [CA314]
Added CA301 -> [CA314, CA301]
Added MA210 -> [CA314, CA301, MA210]
Removed MA210 -> [CA314, CA301]
```

The printed list is bottom to top, so the newest course is on the right.

Example 6 Reversing a string

Push every character, then pop them all: they come out in reverse order.

Reverser.java

```
import java.util.Stack;
public class Reverser {
    public String reverse(String input) {
        Stack<Character> stack = new Stack<>();
        for (int i = 0; i < input.length(); i++)
            stack.push(input.charAt(i));       // J, A, V, A
          String reversed = "";
          while (!stack.isEmpty())
              reversed += stack.pop();                 // A, V, A, J
          return reversed;
      }
}
```

Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner inputObject = new Scanner(System.in);
        System.out.println("Enter String to Reverse: ");

         String input = inputObject.nextLine();
         Reverser test = new Reverser();
         String result = test.reverse(input);
         System.out.println("Reversed String: " + result);
     }
}
```

Output

```
Enter String to Reverse:
JAVA
Reversed String: AVAJ
```

Input ABCDE gives EDCBA ; input JAVA gives AVAJ .

Part C — ArrayStack (stack built on an array)

Example 7 — ArrayStack: full implementation

Our own stack stored in an array. top is both the next free index and the number of elements. When the array is full, it doubles.

ArrayStack.java

```
import java.util.Arrays;
import java.util.EmptyStackException;

public class ArrayStack<T> {
    private final static int DEFAULT_CAPACITY = 2;   // constant
    private int top;          // next free slot = number of elements
    private T[] stack;

      public ArrayStack() {                        // empty stack, default capacity
          this(DEFAULT_CAPACITY);
      }

      public ArrayStack(int initialCapacity) { // empty stack, given capacity
          top = 0;
          stack = (T[]) (new Object[initialCapacity]); // generic array trick
      }

      public void push(T element) {
          if (size() == stack.length)
              expandCapacity();                    // full? double the array
          stack[top] = element;
          top++;
      }

      private void expandCapacity() {
          stack = Arrays.copyOf(stack, stack.length * 2);
      }

      public T pop() throws EmptyStackException {
          if (isEmpty())
              throw new EmptyStackException(); // underflow
          top--;
          T result = stack[top];
          stack[top] = null;
          return result;
      }

      public T peek() throws EmptyStackException {
          if (isEmpty())
              throw new EmptyStackException();
          return stack[top - 1];               // look only, do not remove
      }

      public int size()          { return top; }
      public boolean isEmpty()   { return top == 0; }
      public int capacity()      { return stack.length; }              // added for the trace
}
```

Java cannot do new T[n] , so we create Object[] and cast to T[] .
Bottom of the stack is fixed at index 0. pop() sets the old slot to null so the object can be garbage collected.
capacity() is not on the slides; it is added only so Example 8 can print the array size.

Example 8 — ArrayStack trace: the array grows, then runs out

DEFAULT_CAPACITY = 2. Watch what happens on the third push and on the fourth pop.

ArrayStackTraceDemo.java

```
public class ArrayStackTraceDemo {
    public static void main(String[] args) {
        ArrayStack<String> s = new ArrayStack<>();
        s.push("A");
```

```
        s.push("B");
        System.out.println("size " + s.size() + ", capacity " + s.capacity());
        s.push("C");                      // array full -> doubles to 4
        System.out.println("size " + s.size() + ", capacity " + s.capacity());

          System.out.println("peek: " + s.peek());
          System.out.println("pop: " + s.pop());
          System.out.println("pop: " + s.pop());
          System.out.println("pop: " + s.pop());
          System.out.println("pop: " + s.pop());            // stack empty -> underflow
      }
}
```

Also needs: ArrayStack.java (see Examples 7).

Output

```
size 2, capacity 2
size 3, capacity 4
peek: C
pop: C
pop: B
pop: A
Exception in thread "main" java.util.EmptyStackException
```

Third push: array was full (2/2), so expandCapacity() copied A, B into a new array of size 4.
Fourth pop on an empty stack is underflow: an exception, not a wrong value.
Overflow never happens in this version, because the array doubles instead.

Example 9 Defining our own exception

EmptyCollectionException extends RuntimeException ; the message names the collection, so it can be reused for other collections.

EmptyCollectionException.java

```
public class EmptyCollectionException extends RuntimeException {
    public EmptyCollectionException(String collection) {
        super("The " + collection + " is empty.");
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        try {
            throw new EmptyCollectionException("stack");
        } catch (EmptyCollectionException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
}
```

10. Linked list: traverse the chain

TraverseDemo.java

```
public class TraverseDemo {
     public static void main(String[] args) {
         // Build: head -> 17 -> 92 -> 63 -> 45 -> null
         Node<Integer> head = new Node<>(17);
         Node<Integer> n2 = new Node<>(92);
         Node<Integer> n3 = new Node<>(63);
         Node<Integer> n4 = new Node<>(45);
         head.setNext(n2);
         n2.setNext(n3);
         n3.setNext(n4);            // n4.next stays null = end of list

           Node<Integer> current = head;
           while (current != null) {
               System.out.println(current.getElement());
               current = current.getNext();
           }
       }
 }
```

11. Insert at the front: order matters

ReversedStepsDemo.java

```
public class ReversedStepsDemo {
     static void print(String label, Node<String> head) {
         System.out.print(label);
         Node<String> cur = head;
         for (int i = 0; i < 5 && cur != null; i++) { // max 5 nodes
             System.out.print(cur.getElement() + " -> ");
             cur = cur.getNext();
         }
         System.out.println(cur == null ? "null" : "...");
     }

       public static void main(String[] args) {
           // Correct order
           Node<String> head = new Node<>("B");
           Node<String> newNode = new Node<>("A");
           newNode.setNext(head);    // 1. link
           head = newNode;           // 2. move head
           print("Correct: ", head);

           // Swapped order

           head = new Node<>("B");
           newNode = new Node<>("A");
           head = newNode;           // 2. move head          (B is now unreachable!)
           newNode.setNext(head);    // 1. link               (A points to itself)
           print("Swapped: ", head);
       }
 }
```

12. Node and LinkedStack: full implementation

Node.java

```
public class Node<T> {
     private T element;
     private Node<T> next;

       public Node()            { element = null;    next = null; }
       public Node(T element)   { this.element = element; next = null; }

       public Node<T> getNext()              { return next; }
       public void setNext(Node<T> next)     { this.next = next; }
       public T getElement()                 { return element; }
       public void setElement(T element)     { this.element = element; }
 }
```

LinkedStack.java

```
import java.util.EmptyStackException;
```

```java
public class LinkedStack<T> {
    private int count;
    private Node<T> top;

      public LinkedStack() {
          count = 0;
          top = null;
      }

      public void push(T element) {
          Node<T> temp = new Node<>(element);        // 1. new node
          temp.setNext(top);                         // 2. link it to old top FIRST
          top = temp;                                // 3. then move top
          count++;
      }

      public T pop() throws EmptyStackException {
          if (isEmpty())
              throw new EmptyStackException();
          T result = top.getElement();
          top = top.getNext();                // old top is now unreferenced
          count--;
          return result;


      }

      public T peek() throws EmptyStackException {
          if (isEmpty())
              throw new EmptyStackException();
          return top.getElement();
      }

      public boolean isEmpty() { return count == 0; }
      public int size()        { return count; }
}
```

13. LinkedStack trace

LinkedStackTraceDemo.java

```
public class LinkedStackTraceDemo {
    public static void main(String[] args) {
        LinkedStack<String> s = new LinkedStack<>();
        s.push("A");
        s.push("B");
        s.push("C");                // top -> C -> B -> A -> null
        System.out.println("size: " + s.size());
        System.out.println("peek: " + s.peek());
        System.out.println("pop: " + s.pop());     // C is unlinked
        System.out.println("size: " + s.size());
        System.out.println("peek: " + s.peek());
    }
}
```

14. Bank example with LinkedStack

UndoDepositLinkedDemo.java

```
class BankAccount {
    private double balance;
    // the only change: LinkedStack instead of Stack
    private LinkedStack<Double> history = new LinkedStack<>();

      public BankAccount(double balance) { this.balance = balance; }

      public void deposit(double amount) {
          balance = balance + amount;

          history.push(amount);
          System.out.println("Deposit " + amount + " -> balance " + balance);
      }

      public void undoLastDeposit() {
          if (history.isEmpty()) {
              System.out.println("Nothing to undo");
              return;
          }
          double last = history.pop();
          balance = balance - last;
          System.out.println("Undo " + last + " -> balance " + balance);
      }
}

public class UndoDepositLinkedDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(100);
        acc.deposit(50);
        acc.deposit(20);
        acc.deposit(30);
        acc.undoLastDeposit();
        acc.undoLastDeposit();
    }
}
```

15. Student example with LinkedStack

UndoCourseLinkedDemo.java

```
class Registration {
    private LinkedStack<String> courses = new LinkedStack<>();

      public void addCourse(String code) {
          courses.push(code);
          System.out.println("Added " + code + " | top: " + courses.peek()
                             + " | size: " + courses.size());
      }

      public void undoLastCourse() {
          if (courses.isEmpty()) {
              System.out.println("Nothing to undo");
              return;
```

```
            }
            String removed = courses.pop();
            System.out.println("Removed " + removed + " | size: " + courses.size());
        }
}
public class UndoCourseLinkedDemo {
    public static void main(String[] args) {
        Registration reg = new Registration();
        reg.addCourse("CA314");

            reg.addCourse("CA301");
            reg.addCourse("MA210");
            reg.undoLastCourse();
            reg.undoLastCourse();
        }
}
```